



Code generation with large language models: a survey from neural program synthesis to autonomous software development

Burak Gülmez^{1,2}

Received: 24 November 2025 / Accepted: 24 March 2026 / Published online: 10 April 2026
© The Author(s) 2026

Abstract

Large language models have reshaped code generation, driving a transition from rule-based and statistical methods to transformer-based architectures pretrained on vast code corpora. This survey traces the intellectual lineage from classical program synthesis through pre-transformer neural approaches to contemporary large-scale models, examining code generation capabilities across model architectures, training strategies, task taxonomies, evaluation methodologies, security implications, and ethical considerations. Contemporary models show proficiency in function-level synthesis, program repair, and documentation generation, though performance varies across programming languages and task complexities. Models ranging from 125 M to hundreds of billions of parameters are analyzed (including CodeBERT, GraphCodeBERT, Codex, AlphaCode, CodeGen, StarCoder, CodeLlama, WizardCoder, DeepSeek-Coder-V2, Yi-Coder, and GPT-4) with pass@1 accuracies on HumanEval spanning a wide range across model generations; multi-agent approaches show promise on repository-level and complex benchmarks, though all figures require cautious interpretation given data contamination risks and evaluation protocol differences. Security concerns persist, as models consistently generate vulnerable code across a range of configurations, with vulnerability rates varying substantially depending on model generation, task type, and prompt design. The survey provides critical analysis of architectural design choices, scaling law behavior for code versus natural language, training data curation challenges including legal and ethical dimensions, and the gap between benchmark performance and real-world software engineering workflows. Critical gaps are identified in handling repository-level context, maintaining consistency across extended generation sessions, and providing reliability guarantees. Future trajectories point toward autonomous software engineering agents, hybrid neuro-symbolic verification approaches, and multi-faceted evaluation frameworks, though foundational challenges in correctness verification, security assurance, and trustworthy generation remain unresolved.

Keywords large language models · code generation · program synthesis · software engineering · neural code intelligence · evaluation methodology · code security

1 Introduction

Program synthesis emerged from formal methods research. Early systems relied on logical specifications, constraint solvers, and theorem provers to construct programs that provably satisfied given requirements [29]. These

approaches, while theoretically sound, struggled with scalability. The transition to statistical methods introduced probabilistic reasoning into code generation, though these techniques remained constrained by their reliance on hand-crafted features and limited training data.

Neural approaches fundamentally altered the landscape. Sequence-to-sequence models adapted from machine translation indicated that code could be treated as a structured language amenable to deep learning [16]. The architecture shifted from explicitly encoded program semantics to learned representations extracted from large code corpora. This framework enabled models to capture patterns that defied explicit formalization, albeit at the cost of interpretability and correctness guarantees.

✉ Burak Gülmez
burak.gulmez@ucd.ie

¹ College of Engineering and Architecture, University College Dublin, Dublin, Ireland

² Veta Apt, Dumlupinar Mah. Anafartalar Cad. Cil Sok. No: 3, Nilufer, Bursa 16285, Türkiye

The advent of transformer architectures and pretraining at scale marked another inflection point [25]. Models pretrained on billions of tokens from GitHub repositories, Stack Overflow discussions, and programming documentation acquired broad knowledge of programming patterns, API usage conventions, and common algorithmic solutions. CodeBERT, GraphCodeBERT, and subsequent iterations established that masked language modeling objectives, when applied to code, yielded representations useful across multiple downstream tasks. The field moved from task-specific architectures to general-purpose models fine-tuned for particular applications.

Commercial deployment of code generation systems catalyzed widespread adoption. GitHub Copilot, powered by OpenAI's Codex, reached millions of developers [9]. The tool integrated directly into development environments, offering context-aware suggestions as programmers typed. Acceptance rates reportedly exceeded 30% for suggested completions, though these figures varied across programming languages, project types, and developer experience levels.

This deployment raised questions about productivity impacts, code quality, and developer skill evolution. User studies indicated time savings for routine coding tasks but also revealed concerning patterns: developers sometimes accepted incorrect or insecure suggestions without adequate verification [9]. The shift from writing code entirely manually to steering and validating AI-generated outputs required different skill sets. Whether this represents net positive or negative development remains contested, complicated by the fact that long-term effects on codebase maintainability have not been adequately measured across realistic timescales.

Automated code generation promises to democratize software development. Lowering barriers to programming could enable domain experts to directly implement solutions rather than relying on intermediary developers to translate requirements. Economic implications are considerable; if even modest productivity gains materialize across the industry, the aggregate effect on software development costs could reach billions of dollars annually.

Beyond productivity, code generation systems might improve software quality through consistent application of best practices, though current evidence on this point remains mixed. Models trained on high-quality code repositories could, theoretically, propagate superior coding patterns. However, models also learn from flawed examples, and the mechanisms controlling which patterns dominate in generation remain poorly understood. The tension between learning common patterns (which may include antipatterns) and generating optimal solutions has not been resolved.

This survey addresses four primary questions.

RQ1: What capabilities do current code generation models possess? The answer requires examining performance across diverse tasks, programming languages, and complexity levels.

RQ2: What methods underlie these capabilities? Understanding the architectural choices, training strategies, and prompting techniques that yield effective code generation informs both model development and deployment decisions.

RQ3: How is performance evaluated? Assessment methodologies shape research directions; flawed evaluation can misrepresent progress.

RQ4: What are the limitations? Identifying failure modes, safety concerns, and reliability issues guides future research priorities.

The survey focuses on large language model-based approaches, defined as transformer architectures with parameters numbering in the billions, pretrained on code corpora. Excluded are classical program synthesis methods and purely symbolic approaches. Smaller pre-transformer models (below one billion parameters) are discussed as foundational precursors where they establish concepts central to subsequent large-scale development, but are not the primary focus of analysis. The temporal scope emphasizes developments from 2020 onward, coinciding with the emergence of models that achieved competitive performance on established benchmarks.

More precisely, the survey covers code generation in the general-purpose software engineering sense: function-level synthesis, program completion, code translation, automated repair, and test generation for mainstream programming languages. Domain-specific code generation for areas such as hardware description languages and geospatial computing receives targeted treatment where LLM-based approaches have been applied. Areas with highly specialized code regulation and domain-specific control mechanisms (including safety-critical embedded systems, domain-specific numerical solvers, and real-time control firmware) fall outside the primary scope, as each such domain imposes architectural and regulatory constraints that warrant dedicated surveys.

The boundary between pure LLM-based generation and hybrid neuro-symbolic systems continues to blur. Approaches that integrate formal methods, symbolic reasoning, or constraint-based verification with large language models represent an active frontier [23]. While a full treatment of hybrid systems lies beyond the present scope, this survey acknowledges borderline approaches where relevant, particularly in discussions of correctness guarantees and verification-integrated generation pipelines. Excluding purely symbolic systems does not imply that progress toward reliability guarantees is disregarded; rather, the survey traces

how LLM-centric methods increasingly incorporate formal verification concepts without reverting to classical synthesis frameworks.

This work provides several distinct contributions beyond existing surveys. First, broad coverage of recent models [25, 26] including open-source implementations that enable reproducible research. Second, systematic analysis of evaluation methodologies with critical examination of benchmark limitations, data contamination risks [28], and the gap between controlled evaluation and real-world deployment performance. Third, integration of findings from user studies that assess how developers actually interact with code generation tools, revealing discrepancies between idealized usage scenarios and actual practice patterns [9]. Fourth, treatment of security implications, examining vulnerability introduction rates and defensive strategies [3, 11]. Fifth, a dedicated treatment of ethical considerations including training data copyright, attribution of generated code, and socioeconomic implications for the software engineering profession.

The survey also addresses domain-specific applications: hardware description languages [1, 26], geospatial computing [12], industrial control systems [18], and emerging domains where automated code generation intersects with autonomous systems. Domain-specific applications including autonomous vehicle navigation and unmanned marine vehicle control represent areas where code generation could accelerate development cycles [15, 31], though such applications impose stringent correctness requirements that current LLM-based approaches do not yet reliably satisfy.

The survey proceeds as follows. Section 2 provides background on program synthesis evolution and transformer architectures. Section 3 describes the systematic search methodology and paper selection process. Section 4 surveys code-pretrained language models and their training strategies. Section 5 presents the task taxonomy, methods, and evaluation methodologies in an integrated treatment. Section 6 addresses limitations, security concerns, and ethical considerations. Section 7 discusses future directions and concludes the survey.

2 Background

2.1 Program synthesis: classical approaches

Formal methods approached program synthesis through logical specification. Given preconditions and postconditions expressed in temporal logic or similar formalisms, synthesis algorithms searched the space of possible programs for implementations satisfying the specifications [23]. Deductive synthesis worked backward from desired

outputs to necessary program structures. Inductive synthesis learned from input-output examples, generalizing patterns across instances. Programming by example represented a user-friendly variant where non-programmers provided demonstrations rather than formal specifications.

These approaches guaranteed correctness (when they succeeded) but faced computational intractability for complex specifications. Search spaces grew exponentially with program length. The brittleness to specification errors, where minor imprecision in formal requirements could yield programs wildly divergent from actual intent, limited practical adoption outside specialized domains.

The intellectual lineage from classical synthesis to contemporary LLM-based generation follows a traceable, if non-linear, trajectory. Formal methods established the aspiration of provably correct code; constraint-based synthesis introduced search heuristics that traded completeness for scalability; inductive program synthesis shifted the specification burden from logical formulas to input-output examples. Each transition relaxed guarantees while broadening applicability. The emergence of neural approaches did not invalidate these earlier frameworks so much as reposition the problem: where classical synthesis asked “does this program satisfy a specification,” neural generation asks “does this program appear consistent with patterns observed in training data” [29]. The philosophical distance between these two questions, one grounded in logical entailment and the other in statistical likelihood, defines much of the contemporary tension around code generation reliability. Correctness guarantees, once central to synthesis research, largely vanished when the field adopted probabilistic models; whether and how they might be recovered remains a pressing open question.

2.2 Pre-transformer neural approaches

Before transformer architectures reshaped the field, several neural approaches laid essential groundwork for code generation. Sequence-to-sequence models with attention mechanisms, borrowed from machine translation, first indicated that natural language descriptions could be mapped to code tokens through end-to-end learning. These recurrent architectures suffered from gradient degradation over long sequences and struggled with the rigid syntactic constraints of programming languages; still, they established the viability of treating code generation as a sequence transduction problem.

Neural programmer-interpreters proposed an alternative: rather than generating code as a token sequence, models could learn to manipulate a virtual execution environment, composing primitive operations into programs. This approach better captured the compositional structure

of algorithms but proved difficult to scale beyond simple tasks. Neural module networks pursued a related idea, learning to dynamically assemble task-specific computational graphs from reusable modules. These systems blurred the line between program synthesis and differentiable programming, prefiguring the modular reasoning that would later appear in chain-of-thought prompting strategies for large language models.

Tree-based and graph-based neural models offered structural alternatives to sequential generation. Representing programs as abstract syntax trees enabled models to capture hierarchical properties, potentially learning structural patterns more effectively than sequential models [25]. Tree-based models showed promise for code-to-code transformation tasks where preserving structural relationships proved important. Whether the representational advantages outweighed the increased architectural complexity remained debated; empirical comparisons yielded mixed results depending on task characteristics.

The progression from recurrent sequence models through structured neural representations to attention-based architectures reflects a recurring pattern in the field: each generation of models addressed a specific bottleneck (long-range dependencies, structural awareness, scalability) while introducing new limitations that motivated subsequent architectures.

2.3 Large language models

Transformer architecture provided the foundation for contemporary code generation models. Self-attention mechanisms enabled modeling long-range dependencies without the gradient flow problems that plagued recurrent networks. Pretraining on massive unlabeled corpora through masked language modeling objectives yielded general-purpose representations that transferred to downstream tasks after fine-tuning with task-specific data.

Scaling laws suggested that model performance improved predictably with parameter count, training compute, and dataset size (though the relationships proved more complex than initially hypothesized). Models grew from millions to billions of parameters. GPT-style architectures dominated code generation applications, trained autoregressively to predict the next token given preceding context. This formulation naturally aligned with code completion tasks where models extended partial programs.

The mechanics of how these models encode programming knowledge remains incompletely understood. Representations presumably capture syntactic patterns, API usage conventions, and common algorithmic structures, but the

internal organization of this knowledge and the mechanisms by which it influences generation are largely opaque (creating challenges for debugging model failures and providing guarantees about generated code).

2.4 Hybrid neuro-symbolic approaches

The boundary between purely neural and purely symbolic approaches to code generation has grown increasingly porous. Several lines of research attempt to recombine the reliability of formal methods with the flexibility of learned models. Verification-integrated generation pipelines, for instance, use neural models to produce candidate programs that are then checked against formal specifications; candidates failing verification trigger re-generation or guided repair. Constraint-guided decoding injects syntactic or semantic constraints into the token sampling process, reducing the likelihood of structurally invalid outputs without sacrificing the generative capacity of the underlying model [23]. These hybrid approaches, still in relatively early stages of development, suggest a possible reconciliation between the correctness aspirations of classical synthesis and the scalability of neural generation. Whether they can deliver meaningful guarantees at the scale of real software systems remains to be seen; early results appear promising for narrow, well-specified tasks but have not yet been validated on the kind of large, evolving codebases that characterize industrial software development.

2.5 Software engineering context

Code generation does not occur in isolation. Real software development involves requirements analysis, architecture design, implementation, testing, debugging, documentation, and maintenance across codebases comprising millions of lines spanning hundreds of files with complex interdependencies [37]. Most research evaluates models on isolated function-level tasks, bracketing the broader context. Whether results from these evaluations extrapolate to realistic development scenarios remains an open empirical question.

Modern development practices emphasize version control, continuous integration, code review, and collaborative workflows. How code generation tools integrate into these practices, and whether they disrupt or improve existing processes, depends on factors including tool ergonomics, integration quality, and organizational culture. The technology does not determine outcomes; deployment contexts mediate how tools affect actual development work.

3 Survey methodology

The literature search followed a structured protocol to ensure reproducibility and breadth. Five databases were queried: IEEE Xplore, ACM Digital Library, Scopus, Google Scholar, and arXiv. The primary search string combined terms including “code generation” AND (“large language model” OR “LLM” OR “pretrained language model”) AND (“transformer” OR “neural”). Searches were conducted between January 2024 and June 2025, with backward and forward citation tracking applied to seminal papers identified in initial results.

Inclusion criteria required that studies: (a) addressed code generation or closely related tasks such as program repair, code completion, or code translation; (b) involved transformer-based models with at least one billion parameters or direct derivatives thereof; (c) were published in peer-reviewed venues or deposited on arXiv with sufficient methodological detail from 2020 onward. Exclusion criteria removed studies focused exclusively on classical program synthesis without neural components, natural language generation without code output, and position papers lacking empirical or analytical content.

Initial database queries identified 513 candidate records (387 from database searching and 126 from other sources

including citation tracking and manual identification). After removal of 92 duplicate records, title and abstract screening of 421 unique records excluded 289 papers, yielding 132 articles for full-text assessment. Full-text review against inclusion and exclusion criteria excluded 84 papers (23 not LLM-based, 19 not addressing code generation, 14 outside the temporal scope, 16 with insufficient methodological detail, and 12 duplicate studies), resulting in a final corpus of 48 primary studies that form the analytical backbone of this survey. An additional set of references provides supporting context for historical background, evaluation methodology, and cross-domain perspectives. Figure 1 presents the systematic selection process following PRISMA guidelines.

4 Code-pretrained language models and training

4.1 Early models (2020–2021)

CodeBERT marked an early milestone, adapting BERT’s masked language modeling approach to code [25]. The model was pretrained on code-natural language pairs from GitHub, learning bidirectional representations suitable for understanding tasks. GraphCodeBERT extended this by

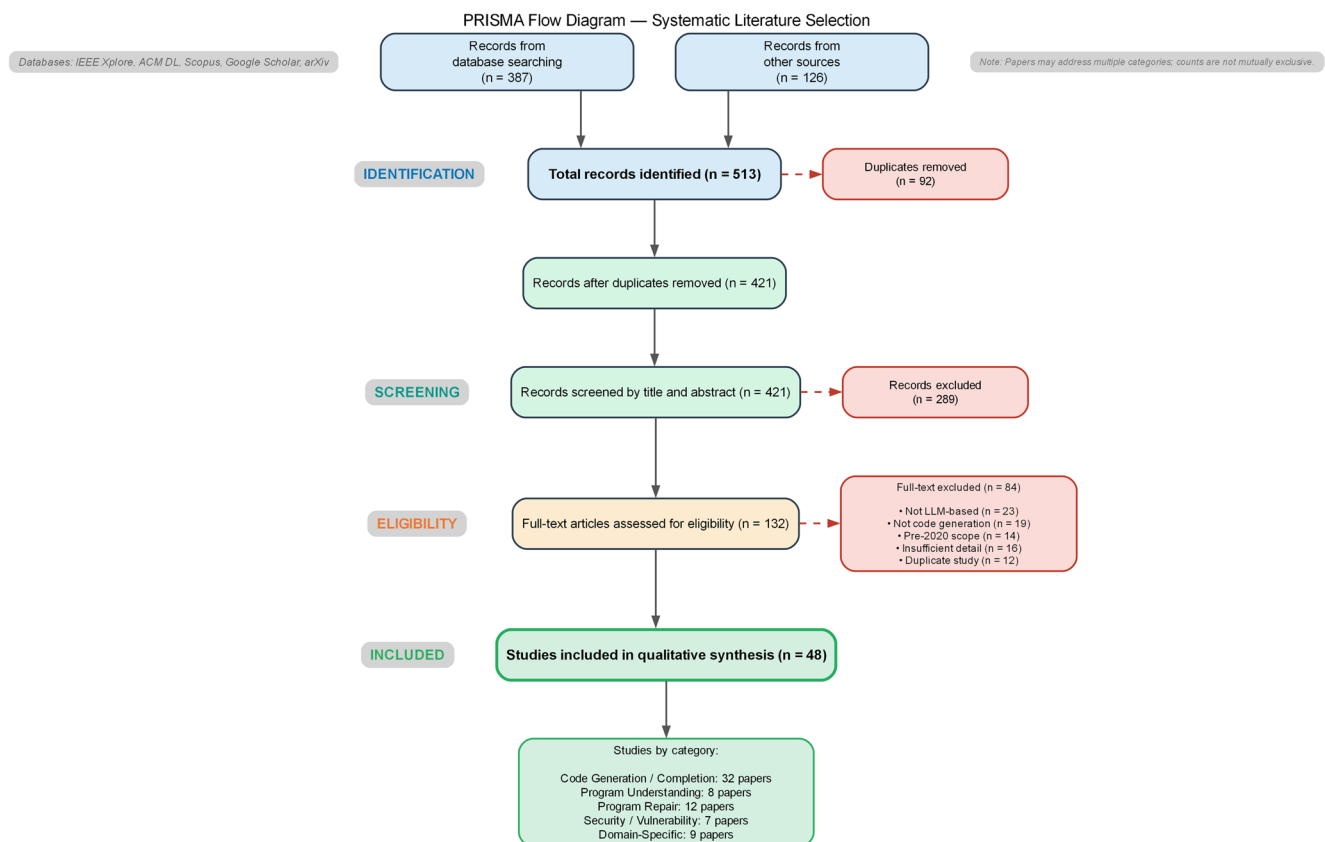


Fig. 1 Paper selection process following PRISMA guidelines

incorporating data flow information into pretraining, representing programs as graphs that capture variable dependency relationships rather than merely syntactic structure. The architectural choice to include data flow graphs was motivated by a specific hypothesis: that understanding how data moves through a program (which variables influence which computations) provides richer semantic signal than token-level or even AST-level representations alone. Empirical results supported this hypothesis for tasks requiring semantic understanding, though the improvement over CodeBERT proved modest for pure generation tasks where left-to-right decoding dominated. CodeT5 adopted an encoder-decoder architecture, enabling both understanding and generation capabilities within a single model. PLBART applied multilingual pretraining across multiple programming languages, investigating whether cross-lingual transfer improved performance on low-resource languages.

These early models established several patterns: pretraining on large code corpora yielded useful representations, incorporating structural information (whether through graph representations or syntax-aware objectives) often improved performance, and bidirectional models excelled at understanding tasks while generation required left-to-right architectures.

4.2 Large-scale models (2021–2022)

Codex represented a qualitative shift [42]. Built on GPT-3 architecture and pretrained on 159 GB of code from public GitHub repositories, it achieved 28.8% pass@1 on HumanEval (a benchmark comprising 164 hand-written programming problems). The scale (12 billion parameters) enabled few-shot learning where the model generated correct solutions from task descriptions and minimal examples, without task-specific fine-tuning.

AlphaCode tackled competitive programming, a domain requiring sophisticated algorithmic reasoning [26]. The model generated massive numbers of candidate solutions, filtering them through test cases. This generate-and-filter approach achieved performance competitive with human programmers at the median level of difficulty in programming competitions, though it remained far from expert performance.

CodeGen investigated scaling effects across models ranging from 350 M to 16B parameters [25]. Results confirmed that larger models generally outperformed smaller counterparts, though returns diminished at the highest parameter counts; the relationship between parameter count and code generation quality appeared to plateau earlier than for natural language tasks, suggesting that code-specific data quality may matter more than raw model size beyond a certain threshold. InCoder introduced fill-in-the-middle

capabilities, a deliberate architectural decision that enabled models to complete code given both prefix and suffix context. This proved more realistic than pure left-to-right generation, since developers frequently insert code into existing functions rather than writing strictly from top to bottom. The fill-in-the-middle objective required modified training procedures but yielded models better aligned with actual editing workflows.

4.3 Open-source models (2022–2024)

The emergence of open-source code models democratized research. StarCoder, trained on The Stack dataset comprising permissively licensed code from GitHub, achieved competitive performance with 15.5B parameters [25]. StarCoder2 improved upon this foundation with better data curation and training strategies. CodeLlama extended Meta's Llama 2, showing that general-purpose language models could be effectively specialized for code through continued pretraining and instruction tuning.

DeepSeek-Coder-V2 achieved particularly strong results, with versions ranging from 16B to 236B parameters [25]. The model reportedly achieved 81.1% pass@1 on HumanEval, approaching GPT-4 performance. These results suggested that open-source models could match or exceed commercial offerings, reducing barriers to research and deployment (though questions about training data composition and evaluation protocols complicate direct comparisons).

WizardCoder applied instruction tuning and evolution techniques to improve code generation. The approach iteratively refined training examples, using the model itself to generate increasingly complex variants. This self-improvement loop yielded performance gains, though concerns about overfitting to evaluation benchmarks arose given the model's involvement in constructing training data. The question of whether such evolved training data genuinely improves generalization or merely teaches models to perform well on specific benchmark formats has not been conclusively resolved.

4.4 Architectural choices and their consequences

The evolution of code-pretrained models reflects a series of architectural bets, each with distinct trade-offs. Encoder-only architectures (CodeBERT, GraphCodeBERT) excelled at understanding tasks but required separate generation components. Encoder-decoder models (CodeT5) offered unified understanding and generation at the cost of increased parameter counts. Decoder-only GPT-style architectures ultimately dominated, driven less by theoretical superiority than by practical alignment with autoregressive generation

and the scaling properties that larger compute budgets demanded. The success of decoder-only models for code generation appears partly contingent on the availability of very large training corpora; at smaller scales, encoder-decoder architectures retained competitive advantages for certain tasks.

Mixture-of-Experts (MoE) architectures, as adopted by DeepSeek-Coder-V2, introduced conditional computation where only a subset of parameters activates for each input token. This approach achieved high benchmark scores with lower inference costs than dense models of equivalent total parameter count. Whether MoE models produce qualitatively different code (perhaps more varied but less consistent) than dense alternatives remains underinvestigated. Scaling laws for code generation also appear to diverge from natural language patterns; several studies report that improvements from increased model size diminish faster for code tasks, possibly because programming languages impose tighter syntactic constraints that can be captured with fewer parameters, while the harder challenge of semantic correctness responds less predictably to scale alone.

4.5 Domain-specific models

Specialization for particular domains or programming languages yielded targeted improvements. SQLCoder focused exclusively on SQL generation from natural language queries [39]. MathCoder targeted mathematical code generation. Models for hardware description languages addressed Verilog and VHDL synthesis [26, 32].

The trade-off between general-purpose and specialized models remains incompletely characterized. Specialized models achieved higher performance on in-domain tasks but lost versatility. For organizations with narrowly defined use cases, this trade-off favored specialization. For general-purpose development tools, broad capabilities proved more valuable despite modestly lower peak performance on any single task.

4.6 Training strategies and data practices

Data collection and filtering affect model quality in ways that are only partially understood. The Stack employed permissive license filtering to address copyright concerns (though legal questions remain unresolved). Data deduplication at scale typically proceeds through MinHash-based approximate matching or exact match detection, removing near-identical code snippets that might skew model behavior toward overrepresented patterns. The impact of deduplication is non-trivial: insufficiently aggressive deduplication allows models to memorize common boilerplate, while overly aggressive deduplication can remove legitimate

repeated patterns (standard library usage, common algorithmic idioms) that models should learn. Quality filtering introduces additional biases; heuristics based on linting scores, documentation presence, and test coverage tend to favor well-maintained open-source projects, potentially underrepresenting code styles common in industrial or rapid-prototyping contexts. Filtering based on repository star counts, a widely used proxy for quality, systematically excludes code from less popular but technically sound repositories, introducing a popularity bias whose downstream effects on model behavior remain poorly quantified.

Training objectives evolved beyond basic language modeling. Fill-in-the-middle training enabled bidirectional context use [25]. Instruction tuning aligned models with user intents by fine-tuning on examples pairing natural language descriptions with corresponding code implementations. Reinforcement learning from human feedback (RLHF) refined model outputs based on preferences for code quality, security, and stylistic considerations, though the resource intensity limited widespread adoption.

4.7 Legal and ethical dimensions of training data

The legal status of training on publicly available code remains contested and varies across jurisdictions. Large-scale scraping of GitHub repositories raises questions about copyright compliance: code published under restrictive licenses (GPL, AGPL) may impose obligations on derivative works, and whether model outputs constitute derivative works is an unresolved legal question. The Stack dataset addressed this partially through opt-out mechanisms and permissive license filtering, but such approaches do not eliminate ambiguity. Attribution presents a separate challenge; models generate code that may closely resemble specific training examples without crediting original authors, creating tensions with open-source norms that emphasize provenance and attribution. These legal uncertainties carry practical consequences: organizations deploying code generation tools face liability risks that are difficult to quantify, and individual developers whose code appears in training data without explicit consent have limited recourse under current legal frameworks.

Table 1 presents a comparison of major code language models across key dimensions. These figures should be interpreted cautiously. Benchmark scores reflect specific evaluation conditions and may be inflated by data contamination, where models encountered test problems during pretraining. The improvements from 28.8% (Codex) to approximately 85.4% (Yi-Coder) on HumanEval likely reflect a combination of increased scale, better training data curation, architectural refinements, and potential evaluation leakage; disentangling these contributions remains an open

Table 1 Code LLMs comparison

Model	Parameters	Architecture	Training Data Size	Languages	Pass@1 (HumanEval)	Year	Open Source
CodeBERT	125 M	RoBERTa	6.4 M functions	6	N/A (encoder only)	2020	Yes
GraphCodeBERT	125 M	Transformer + DFG	2.3 M functions	6	N/A (encoder only)	2021	Yes
Codex	12B	GPT	159 GB	Python	28.8%	2021	No
AlphaCode	41B	Encoder-Decoder Transformer	715 GB	Multiple	N/A*	2022	No
CodeGen-16B	16B	GPT	450 GB	Multiple	29.3%	2022	Yes
StarCoder	15.5B	GPT	1 TB	80+	33.6%	2023	Yes
CodeLlama	7B–70B	Llama 2	500B tokens	Multiple	48.8% (34B variant)	2023	Yes
DeepSeek-Coder-V2	236B (21B active)	MoE	6 T tokens	Multiple	81.1% (reported)	2024	Yes
GPT-4	Undisclosed	GPT	Unknown	Multiple	~67% (reported)	2023	No
Yi-Coder	9B	Yi	2.4T tokens	Multiple	85.4% (reported)	2024	Yes
WizardCoder	15B	StarCoder-based	Evolved dataset	Multiple	57.3%	2023	Yes
RTL-Coder	6.7B	DeepSeek-based	HDL-focused	Verilog/VHDL	34% (VerilogEval)	2024	Yes

*AlphaCode was evaluated on Codeforces competitive programming (~34% solve rate); HumanEval pass@1 not reported

methodological challenge. Direct comparison across models is further complicated by differences in evaluation protocols, temperature settings, and sampling strategies.

Figure 2 presents a timeline visualization of major model releases from 2020 to 2025, illustrating the acceleration in both model scale and reported performance.

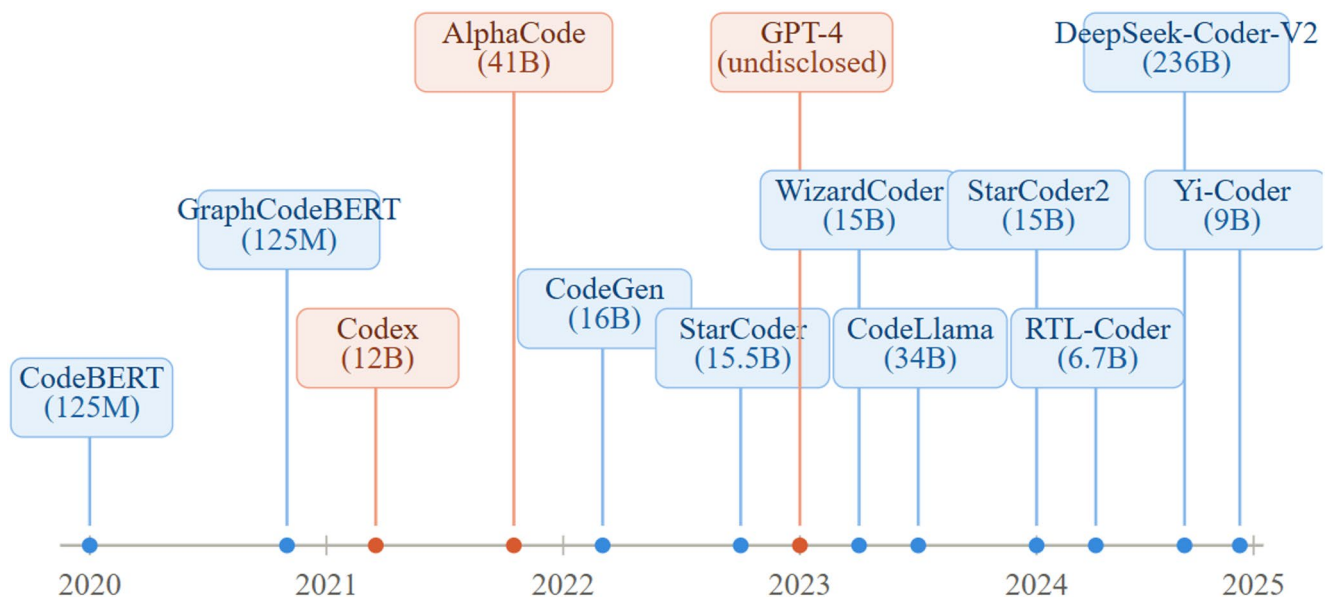
5 Task taxonomy, methods, and evaluation

This section integrates the discussion of code generation tasks, the techniques applied to address them, and the methodologies used to measure progress. Treating these topics together reflects their deep interdependence: task definitions shape which methods are developed, methods constrain

what can be evaluated, and evaluation protocols in turn influence which tasks attract research attention.

5.1 Task taxonomy

Code generation encompasses diverse activities [35]. Generation tasks include code completion (predicting subsequent tokens given context), function synthesis (generating complete functions from specifications), program generation (producing multi-function programs), and test generation (creating test cases for existing code). Understanding tasks comprise code summarization, documentation generation, code search, and clone detection. Transformation tasks involve code translation between languages, refactoring, optimization, and modernization of legacy codebases.

**Fig. 2** Code LLM release timeline, 2020–2025

Quality assurance tasks address bug detection, program repair, vulnerability detection, and code review automation. Interactive tasks enable code explanation, question answering about code, and debugging assistance.

This taxonomy proves useful for organizing research but obscures overlaps. Code completion and synthesis exist on a continuum; the boundary between completing a partially-written function and synthesizing an entire function from a docstring is not sharp. Similarly, program repair combines understanding (identifying bugs), generation (producing fixes), and quality assurance (verifying corrections).

The distribution of research effort across these categories remains uneven [8]. Function-level generation dominates due to convenient evaluation protocols and established benchmarks. Repository-level tasks, despite greater practical importance, receive less attention due to evaluation challenges. Interactive tasks, though potentially reshaping developer workflows, remain underinvestigated because measuring effectiveness requires expensive human evaluation (Table 2).

The unevenness in research distribution reflects more than mere benchmark convenience. Function-level

generation attracted disproportionate attention because it aligned naturally with the autoregressive generation framework, produced results amenable to automated evaluation, and offered clear commercial applications through tools like GitHub Copilot. Repository-level and interactive tasks, by contrast, require contextual reasoning that exceeds current model capacities and evaluation frameworks that do not yet exist in mature form. This bias toward tractable problems creates a risk: the field may optimize for tasks that are measurable rather than tasks that matter most for practical software development. The overlap between task categories further complicates the picture; retrieval-augmented code generation, for instance, simultaneously involves understanding (to determine what context to retrieve), generation (to produce code), and implicitly, quality assurance (to verify that retrieved context was appropriately integrated).

Figure 3 presents a visual taxonomy of code generation task categories and their interrelationships.

5.2 Methods and techniques

5.2.1 Function-level synthesis

Natural language to code generation represents the prototypical application. Given a docstring or natural language description, models generate function implementations [16, 21]. The HumanEval benchmark established a standard evaluation protocol: 164 programming problems with function signatures, docstrings, and unit tests. Models generate candidate implementations; correctness is assessed by executing tests.

Performance on this benchmark improved dramatically between 2021 and 2024. Codex achieved 28.8% pass@1 (fraction of problems solved correctly on first attempt). GPT-4 reached approximately 67%. DeepSeek-Coder-V2 and Yi-Coder reported ~ 81.1% and ~ 85.4% respectively [25]. These figures, frequently reported as evidence of rapid progress, require careful qualification. Concerns about data contamination persist [28], and models may have encountered HumanEval problems or semantically similar variants during pretraining. When performance numbers are drawn from secondary sources rather than controlled replications, differences in temperature settings, sampling strategies, and prompt formatting can inflate apparent performance gaps between models.

Code quality beyond mere correctness receives less attention. Generated code may compile and pass tests while exhibiting poor style, unnecessary complexity, or inefficiency [25]. Human evaluation reveals that even correct solutions often contain code smells or violate best practices.

Table 2 Task distribution in literature

Task Category	Number of Papers	Percentage	Primary Evaluation Metric
Code Completion/Generation	32	66.7%	Pass@k, CodeBLEU
Program Understanding	8	16.7%	BLEU, ROUGE, human eval
Code Translation	6	12.5%	Compilation success, functional equivalence
Program Repair	12	25.0%	Fix success rate, test pass rate
Test Generation	5	10.4%	Coverage metrics, mutation score
Security/Vulnerability	7	14.6%	Vulnerability detection rate, CWE coverage
Domain-Specific (HDL, Geospatial, etc.)	9	18.8%	Domain-specific metrics
Evaluation/Benchmarking	11	22.9%	N/A (meta-analysis)
Prompt Engineering/Techniques	13	27.1%	Improvement over baseline

Papers often address multiple task categories; percentages sum to more than 100%. The concentration on generation tasks reflects both the maturity of evaluation methodologies and the commercial incentives driving research investment in this area

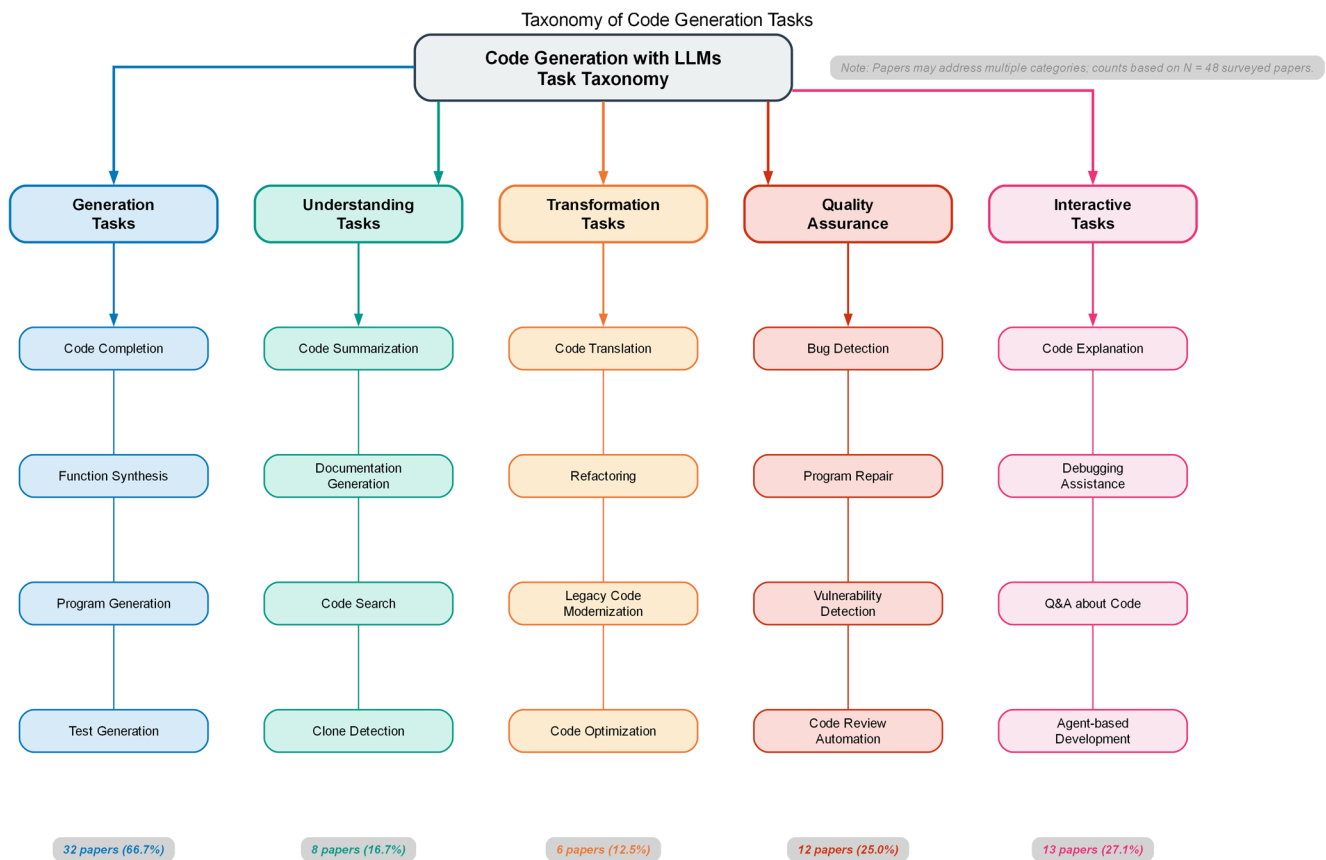


Fig. 3 Code generation task taxonomy

5.2.2 Prompt engineering

Prompt design affects performance in ways that are sometimes counterintuitive [2, 10]. Few-shot prompting, where examples of similar problems and solutions precede the target problem, can improve results though gains vary. Chain-of-thought prompting, successful in natural language reasoning tasks, applies to code generation by explicitly generating intermediate reasoning steps before producing code [20].

The ARCHCODE framework showed that incorporating explicit preconditions and postconditions as design constraints during prompting improved generation accuracy [10]. This approach bridges formal methods and LLM generation, providing structured specifications that guide the model. Similarly, the symmetry-aware approach proposed by Li and Lyu [19] distills pseudocode reasoning into code generation, improving performance for lightweight models.

Self-debugging represents another prompting strategy [16]. Models generate initial code, execute it against test cases, observe failures, and iteratively refine the implementation. This approach improved pass rates but incurred computational costs from multiple generation cycles. The effectiveness depends on the model's ability to accurately

diagnose errors from test failures, a capability that varies across models.

5.2.3 Retrieval-augmented generation

Real code exists within repository context [36, 39]. Functions call other functions, import external libraries, inherit from classes defined elsewhere. Models that ignore this context generate code that may appear plausible in isolation but fails when integrated into the broader codebase.

Retrieval-augmented generation addresses this by incorporating relevant code snippets from the repository into the prompt [39]. Given a generation task, a retrieval system identifies potentially relevant functions, classes, or documentation. These are prepended to the prompt, providing context for generation. The Geo-FuB framework for geospatial code generation constructed an operator-function knowledge base from 154,075 Google Earth Engine scripts [12], reducing hallucinations by 31–34% compared to unaided generation. Similarly, the APICoder framework improved API service invocation code generation through targeted retrieval of API documentation and usage patterns [38].

The challenge lies in retrieving appropriate context. Simple similarity-based retrieval may surface irrelevant code. More sophisticated approaches use the model itself to identify what context it needs, though this introduces circularity. Current retrieval systems also face a scalability problem: as repository size increases, the ratio of relevant to irrelevant retrievable context decreases, demanding increasingly precise retrieval mechanisms that do not yet exist for code at production scale.

5.2.4 Execution-guided generation

Executing partial code and using runtime feedback to guide generation improves correctness [17]. The ROCODE system integrates backtracking mechanisms with program analysis during generation [17]. When incremental error detection identifies issues, the model backtracks and attempts alternative generation paths, eliminating errors early rather than post-processing.

The muFiX framework addresses specification misunderstanding by combining thought-eliciting prompting with feedback-based refinement [33]. The system first prompts models to analyze test cases and generate specification understanding, then uses execution feedback to identify and correct misunderstandings. Evaluations showed 35.62% average improvement in pass@1 compared to direct generation.

5.2.5 Multi-agent systems

Collaborative code generation using multiple LLM agents represents an emerging approach [22, 37]. The GeoColab framework defines three agent roles (product manager, algorithm engineer, and programmer) that interact to generate geospatial code [37]. This division of labor improved code executability and accuracy by 7.59–26.09% compared to single-agent generation.

Similar multi-agent approaches have been applied to general software engineering [22]. The SOEN-101 framework emulates software process models (Waterfall, TDD, Scrum) using specialized agents embodying different development roles. Evaluations showed that the Scrum-based approach achieved highest performance, with 75.2% pass@1 on HumanEval. The effectiveness of multi-agent systems depends on coordination mechanisms, agent specialization, and communication protocols.

5.3 Evaluation methodologies

5.3.1 Benchmarks

HumanEval established the dominant evaluation framework [4]. The benchmark comprises 164 hand-written programming problems primarily in Python. Each problem includes

a function signature, docstring, implementation, and unit tests. Models generate candidate implementations; correctness is assessed via test execution.

MBPP (Mostly Basic Python Problems) contains 974 problems spanning a range of difficulties [35, 41]. APPS (Automated Programming Progress Standard) addresses competitive programming with 10,000 problems across difficulty levels [43]. CodeXGLUE provides a multi-task benchmark encompassing generation, understanding, and transformation tasks.

Multilingual evaluation uses MultiPL-E, which translates HumanEval and MBPP problems into 18 programming languages [24]. Results reveal considerable variation; models perform best on Python (over-represented in training data) and progressively worse on lower-resource languages.

Domain-specific benchmarks assess specialized capabilities. DS-1000 evaluates data science code generation. Spider tests text-to-SQL generation. VerilogEval measures hardware description language synthesis [26]. These benchmarks often reveal weaknesses obscured by performance on general-purpose datasets.

Figure 4 presents a comparative visualization of benchmark performance across representative models. Representative pass@1 scores across models and benchmarks are presented in Table 3.

5.3.2 Metrics and their limitations

Pass@k remains the primary correctness metric [5]. Pass@1 measures the fraction of problems where the first generated solution passes all tests. Pass@10 and Pass@100 measure whether at least one of multiple generated solutions is correct. The metric, while convenient and widely adopted, fails to capture several quality dimensions that matter for practical deployment. A model achieving high pass@10 but low pass@1 generates many incorrect solutions along with occasional correct ones; whether this pattern proves useful depends entirely on deployment context. More fundamentally, pass@k says nothing about code quality, security, or maintainability. A solution that passes unit tests may contain injection vulnerabilities, use deprecated APIs, exhibit quadratic complexity where linear solutions exist, or violate established coding conventions. The metric also ignores integration complexity: code that functions correctly in isolation may fail when embedded in a larger system with conflicting dependencies or implicit assumptions. The field's reliance on pass@k as the primary yardstick has arguably directed optimization effort toward a narrow notion of correctness at the expense of the broader quality attributes that characterize production-ready code.

CodeBLEU extends BLEU (from machine translation) to code by incorporating abstract syntax tree matching, data

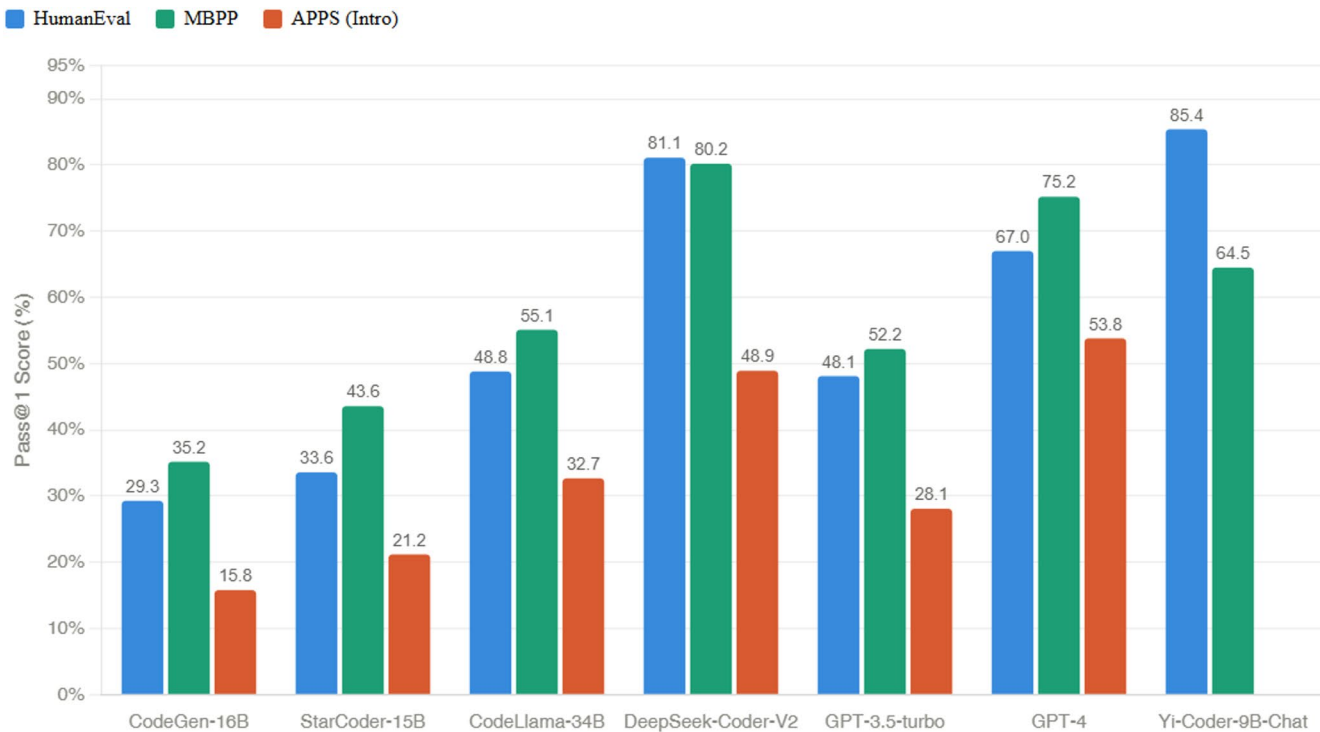


Fig. 4 Benchmark performance comparison across models

Table 3 Representative benchmark performance (Pass@1 Scores)

Model	HumanEval	MBPP	APPS (Intro)	DS-1000	VerilogEval	Spider
CodeGen-16B	29.3%	35.2%	15.8%	N/A	N/A	N/A
StarCoder-15B	33.6%	43.6%	21.2%	N/A	22.1%	N/A
CodeLlama-34B	48.8%	55.1%	32.7%	38.4%	N/A	N/A
DeepSeek-Coder-V2	81.1%	80.2%	48.9%	54.1%	N/A	80%
GPT-3.5-turbo	48.1%	52.2%	28.1%	35.7%	28.9%	62.3%
GPT-4	67.0%	75.2%	53.8%	49.2%	41.7%	83.5%
Yi-Coder-9B	85.4%	64.5%	N/A	N/A	N/A	N/A
RTL-Coder-6.7B	N/A	N/A	N/A	N/A	34.0%	N/A

flow graph similarity, and n-gram overlap [25]. The metric correlates better with human judgments than token-level metrics but remains an imperfect proxy for code quality.

Execution-based metrics assess functional correctness but cannot verify security properties or absence of side effects [4]. Property-based testing offers stronger correctness characterization by checking that generated code satisfies formal properties such as idempotence and monotonicity. Initial studies suggest models struggle with property satisfaction even when passing unit tests. The development of multi-faceted evaluation suites that combine functional correctness, security analysis, performance profiling, and maintainability assessment appears necessary; yet such suites face the practical obstacle of being expensive to construct and difficult to standardize across research groups.

Human evaluation remains the gold standard but scales poorly [9]. Developers assess generated code for correctness,

readability, maintainability, and adherence to conventions. These judgments capture quality dimensions missed by automated metrics. User studies reveal that developers rate even high-performing models as producing code requiring considerable revision in 30–40% of cases.

5.3.3 Data contamination

Benchmark contamination threatens evaluation validity [28]. If models encounter test problems during pretraining, they may memorize solutions rather than showing genuine generalization. Detecting contamination is challenging; models could have seen semantically similar problems even if exact matches are absent from training data.

Riddell et al., [28] conducted systematic contamination analysis, finding considerable overlap between popular benchmarks and code available in public repositories

potentially included in training data. The study quantified contamination at both surface (n-gram overlap) and semantic (solution similarity) levels. Results showed that models perform markedly better on contaminated subsets, suggesting inflated performance estimates.

Mitigation strategies include constructing new benchmarks post-training (though this provides only temporary relief), continuous evaluation on freshly created problems, and holdout-based approaches where benchmark problems are withheld from public repositories prior to model training. The VerilogEval study tracked model performance over time as new problems were added, observing performance degradation when models faced genuinely novel problems [26]. Each mitigation strategy carries limitations: post-training benchmarks become contaminated as soon as they are published, continuous evaluation demands ongoing human effort to create problems, and holdout approaches face scalability constraints when training corpora encompass most publicly available code. No single approach adequately resolves the contamination problem, and the field has not yet converged on a standard protocol for contamination-aware evaluation.

5.3.4 Benchmark Performance Versus Real-World Utility

The gap between benchmark scores and practical value deserves direct confrontation. Benchmarks emphasize isolated function-level tasks with complete specifications and well-defined test suites. Real software development involves incomplete or ambiguous requirements, complex cross-file dependencies, evolving codebases, and collaborative workflows where multiple developers modify interrelated components simultaneously. A model achieving 85% pass@1 on HumanEval may perform markedly differently when asked to generate code that must integrate with an existing module, maintain consistency with project conventions, handle edge cases not captured in brief specifications, or support long-term maintainability. Cross-file consistency (where generated code must respect type contracts, naming conventions, and architectural patterns established across a large codebase) remains almost entirely unevaluated by current benchmarks. The same holds for collaborative development practices: how generated code affects code review workloads, merge conflict rates, and team coordination has received minimal empirical study (Table 3).

Performance varies across benchmarks, programming languages, and task complexities. General-purpose models excel on Python tasks (over-represented in training) but struggle with domain-specific languages. Specialized models (like RTL-Coder for hardware) outperform larger general-purpose models in their domains despite smaller parameter counts.

6 Limitations, security, and ethical considerations

This section consolidates the discussion of technical limitations, security vulnerabilities, and ethical dimensions that collectively constrain the reliability and responsible deployment of LLM-based code generation systems.

6.1 Technical limitations

Context length constraints limit the complexity of code that models can generate [40]. Transformer attention mechanisms scale quadratically with sequence length, imposing practical limits around 8,000–32,000 tokens for most models. Repository-level code generation requires reasoning over codebases orders of magnitude larger. Proposed solutions including hierarchical attention, sparse attention, and retrieval-augmented approaches partially address this but introduce their own complications.

Reasoning limitations manifest when tasks require complex algorithmic reasoning, mathematical insight, or subtle logical inference [34]. Models excel at pattern matching and surface-level synthesis but struggle with novel problem-solving requiring genuine understanding. The boundary between memorization and generalization remains unclear; models may succeed on problems similar to training examples while failing on slightly novel variants.

Long-term dependency modeling proves challenging. Code often exhibits dependencies spanning hundreds or thousands of lines (variable definitions affecting later usage, architectural patterns constraining implementation choices). Models effectively capture local patterns within attention windows but lose coherence over longer spans.

6.2 Quality and correctness issues

Generated code passing unit tests does not guarantee correctness [4, 34]. Tests may have incomplete coverage, miss edge cases, or fail to verify important properties. Property-based testing reveals that 30–32% of solutions passing unit tests fail to satisfy fundamental correctness properties. Another 18–23% fail outright on property tests.

Code quality issues include unnecessary complexity, poor variable naming, inadequate comments, and violations of language-specific idioms [25]. While these rarely cause outright failures, they impair maintainability and comprehensibility. Human evaluation consistently rates generated code lower than human-written code on these dimensions.

Error patterns analysis reveals that models make non-trivial, multi-line errors distributed across function implementations [34]. Common error types include incorrect loop boundaries, mishandled edge cases, improper exception

handling, and logical errors in conditional branches. These errors often require understanding the overall function logic to diagnose and fix, making automated repair challenging.

6.3 Security vulnerabilities and defensive strategies

Models generate vulnerable code at concerning rates [3, 11]. The CoSEFA study found that even GPT-4 produced code containing API misuses in 62% of cases. Such misuses can lead to resource leaks, program crashes, or security vulnerabilities when deployed in production systems.

Common vulnerability categories include SQL injection, cross-site scripting, buffer overflows, and cryptographic misuse [3]. Prompt engineering to emphasize security considerations (e.g., explicitly requesting secure code, providing security-focused examples) reduces but does not eliminate vulnerability introduction. The Syntax Mitigation Prompt reduced syntax errors by 25–71% across models but security vulnerabilities persisted.

Backdoor attacks through prompt engineering represent an emerging threat [27]. The BadCodePrompt study showed that adversarial examples injected via few-shot prompting could cause LLMs to generate code with hidden backdoors. Attack success rates reached 98.53% for GPT-4 on certain tasks. Existing defense mechanisms proved largely ineffective, highlighting the need for security verification beyond automated testing.

LLMs reproduce vulnerabilities present in training data. Code repositories contain flawed examples; models learn both correct and incorrect patterns [3]. Without mechanisms to distinguish good from bad code, models generate vulnerable implementations at rates reflecting training distribution. GPT-4 with security-focused prompts still produced vulnerable code in approximately 30% of cases.

Automated vulnerability detection tools can identify some generated vulnerabilities [11]. The CoSEFA system integrates static analysis to detect API misuses in generated code. Static analysis has known limitations (false positives, incomplete coverage); relying solely on automated checking provides insufficient security assurance.

Supervised co-decoding offers one mitigation approach [11]. A small security-focused model fine-tuned on code changes fixing vulnerabilities guides the generation of larger base models. The approach improved generation security by 9.34% while maintaining functional correctness. This requires maintaining separate security models and increases inference costs.

Recent work on adversarial robustness has revealed additional dimensions of the security problem. Dora et al., [7] conducted a security-centric evaluation of LLM-generated

web application code, finding that models consistently introduced vulnerabilities in authentication flows, session management, and input validation routines even when explicitly prompted for secure implementations. The persistence of these vulnerabilities across different models and prompting strategies suggests that security-aware generation requires deeper architectural interventions rather than surface-level prompt modifications. Cross-domain perspectives from observer-based verification systems [6, 30] offer conceptual parallels: just as observer-based approaches in control systems provide runtime bounds and fault detection guarantees through mathematical formalization, analogous verification layers for code generation could provide runtime assurance that generated code satisfies security invariants. While direct technical transfer between these domains remains limited, the underlying principle (that safety-critical systems require formal verification layers independent of the generative mechanism) applies with equal force to LLM-produced code.

6.3.1 Actionable security guidance

For practitioners deploying code generation tools in production environments, several concrete recommendations emerge from the literature. Code generation should be restricted or subjected to mandatory human review in security-sensitive contexts: authentication logic, cryptographic operations, input validation, database access layers, and network-facing components. Organizations should implement systematic human-in-the-loop enforcement through integration with existing code review workflows, where generated code is flagged for priority review by security-trained developers. Security-aware training and inference strategies, including supervised co-decoding and security-focused fine-tuning, should be evaluated for production deployment; the modest inference cost increase (typically 15–25% overhead for co-decoding approaches) may be justified by the reduction in vulnerability introduction rates. Automated static analysis should be applied as a minimum baseline, with the understanding that it catches only a subset of potential vulnerabilities. Organizations should treat LLM-generated code with the same (or greater) scrutiny applied to code from untrusted external contributors.

6.4 Ethical and broader considerations

The ethical dimensions of LLM-based code generation span training data provenance, output attribution, bias propagation, and socioeconomic impacts. Training data copyright remains legally ambiguous: models trained on copyrighted

code may generate outputs that closely resemble specific copyrighted implementations, raising questions about fair use and derivative work doctrine that courts have not yet resolved. Attribution of generated code presents a related challenge; when a model produces code that functionally replicates a specific open-source implementation, the absence of attribution violates the norms (if not always the legal requirements) of the open-source community.

Bias amplification constitutes another concern. Models trained predominantly on code from certain communities, languages, and coding styles may propagate and reinforce existing biases in software development practices. Code repositories over-represent certain demographic groups, programming languages, and software domains; models trained on this data inherit and potentially amplify these imbalances. The socioeconomic implications for the software engineering profession are considerable: if code generation tools reduce demand for routine programming tasks, junior developers who traditionally build skills through such tasks may face altered career trajectories. Whether this shift ultimately benefits the profession (by freeing developers for higher-level design and architecture work) or harms it (by eroding the learning pathway from novice to expert) remains contested and likely depends on how organizations integrate these tools into their development workflows.

6.5 Cross-domain reliability perspectives

Insights from deep learning applications in other high-stakes domains inform the discussion of code generation reliability. In medical imaging, rigorous evaluation frameworks have been developed to ensure that deep learning systems meet trustworthiness standards before clinical deployment. Iqbal et al., [14] developed an end-to-end deep convolutional neural network for identifying human induced pluripotent stem cell-derived endothelial cells, a task where misclassification carries direct consequences for patient safety. Similarly, Iqbal et al., [13] addressed automated identification of gastrointestinal tract abnormalities using deep convolutional neural networks with endoscopic images, emphasizing the need for evaluation protocols that go beyond aggregate accuracy metrics to capture clinically relevant failure modes. These works reveal a pattern applicable to code generation: in domains where errors carry meaningful consequences, evaluation must move beyond summary performance statistics to capture the distribution, severity, and recoverability of failures. The code generation community could benefit from adopting similar evaluation rigor, particularly as generated code increasingly enters production systems where bugs and vulnerabilities impose real costs.

6.6 Deployment challenges

Real-world deployment of code generation tools introduces practical challenges that controlled evaluations do not capture. Integration into existing development pipelines requires compatibility with version control systems, continuous integration platforms, and code review tools. Developer trust calibration represents a recurring problem: studies indicate that developers sometimes over-rely on generated suggestions (accepting insecure or incorrect code without adequate review) while at other times under-rely on them (rejecting correct suggestions due to general skepticism about AI-generated code). Organizational adoption patterns vary markedly; teams with strong code review cultures appear to integrate generation tools more safely than teams where review practices are weak, suggesting that the technology's safety profile depends as much on organizational context as on model capability.

Security-aware training data curation could reduce vulnerability generation [3]. Filtering training corpora to remove vulnerable code might prevent models from learning insecure patterns. The challenge lies in identifying vulnerabilities at scale; automated tools have high false positive rates, while manual review is prohibitively expensive for the massive datasets used in LLM pretraining.

Human-in-the-loop verification remains essential for security-critical applications. Generated code should undergo security review and testing before deployment. This reduces but does not eliminate the productivity benefits that motivated adopting code generation tools initially. The optimal balance between automation and human oversight varies by risk tolerance and domain.

7 Future directions and conclusion

7.1 Research priorities and emerging trajectories

The trajectory points toward increasingly capable and autonomous systems. Near-term advances (1–3 years) will likely deliver longer context windows through architectural innovations like sparse attention or hierarchical processing [40]. This enables better repository-level understanding. Improved reasoning capabilities through techniques like chain-of-thought, self-refinement, and multi-agent collaboration should improve performance on complex tasks.

Medium-term prospects (3–5 years) envision autonomous software engineering agents capable of full-stack development from high-level specifications [37]. Such systems would handle requirements analysis, architecture design, implementation, testing, and documentation with minimal human intervention. Self-improving codebases

that automatically refactor and optimize themselves represent another possibility, though challenges in correctness verification and maintaining human comprehensibility must be addressed.

Research priorities include formal verification integration to provide correctness guarantees [23]. Combining neural code generation with formal methods could yield systems that generate provably correct code for critical applications. Better evaluation methodologies that capture real-world development complexity beyond isolated function generation are needed. Human-AI collaboration models that optimize the division of labor between humans and AI remain underinvestigated [9].

Domain-specific applications will continue expanding. Hardware design, scientific computing, robotics control, and other specialized domains can benefit from tailored code generation systems [18, 26]. The economic incentives for improving developer productivity ensure continued investment, though questions about long-term effects on software quality, developer skill evolution, and employment patterns remain open.

7.2 Distinguishing recent advances

What distinguishes the current generation of code generation systems from their predecessors is not merely improved benchmark scores but a qualitative shift in capability and deployment context. Earlier neural code generation systems operated primarily as research prototypes, evaluated on curated datasets by their developers. Current systems are deployed at scale, used daily by millions of developers, and integrated into commercial products that generate revenue. This transition from laboratory to production exposed failure modes that benchmarks do not capture: hallucinated API calls, inconsistent naming conventions within generated blocks, subtle logical errors that pass unit tests but fail integration tests. The field has moved from asking “can models generate code?” to asking “can models generate code that is safe, maintainable, and trustworthy enough for production use?” This latter question remains largely unanswered.

The gap between what current models achieve and what autonomous software engineering would require is considerable. Generating correct isolated functions differs qualitatively from maintaining coherent architectural decisions across a large codebase, managing evolving requirements, or reasoning about system-level properties like performance, security, and reliability. Closing this gap demands not only larger or better-trained models but fundamentally new approaches to context management, long-horizon planning, and verification that can operate at the scale and complexity of real software projects.

7.3 Concluding synthesis

Code generation with large language models has progressed rapidly. Models achieve competitive performance on established benchmarks, generate useful suggestions in commercial deployment, and show potential across diverse application domains. The technology has transitioned from research curiosity to practical tool used by millions of developers daily.

Limitations persist. Models struggle with complex reasoning, generate vulnerable code, exhibit unreliable behavior on novel problems, and perform inconsistently across programming languages. The gap between controlled evaluation and deployment performance, combined with data contamination concerns, suggests that published results may overstate capabilities. Security risks, legal uncertainties around training data and generated code ownership, and questions about long-term effects on software quality warrant continued scrutiny.

The field stands at a point where further scaling alone appears unlikely to resolve the deepest remaining challenges. Architectural innovations, better training strategies, integration of formal verification, and more sophisticated prompting techniques all contribute partially, but no single approach addresses the full spectrum of limitations identified in this survey. Autonomous software engineering remains aspirational; current systems augment rather than replace human developers.

Several unifying themes emerge from this survey. First, the tension between scalability and correctness guarantees, inherited from the classical synthesis tradition, persists in neural approaches and may require hybrid neuro-symbolic solutions. Second, evaluation methodologies have not kept pace with model capabilities; the field urgently needs benchmarks that capture real-world software engineering complexity rather than isolated function generation. Third, security and ethical considerations are not peripheral concerns but central constraints on responsible deployment. Fourth, the integration of code generation tools into development workflows depends as much on organizational and human factors as on model performance.

Research should prioritize correctness verification, security assurance, better evaluation methodologies that capture real-world complexity, and human-AI collaboration models. Understanding how models represent and reason about code internally could inform development of more reliable and interpretable systems.

The ultimate goal, systems that make software development more accessible while maintaining high quality and security standards, requires addressing technical, social, and ethical dimensions in concert. The transformation of software engineering through AI-assisted code generation has

begun but remains incomplete. Realizing the technology's potential while mitigating its risks will shape the trajectory of software development for the coming decades, demanding sustained collaboration across machine learning, software engineering, formal methods, and policy research communities.

Funding Open Access funding provided by the IReL Consortium. No funding received.

Data availability There is no data in this paper.

Declarations

Generative AI usage Claude Sonnet 4.5 was used for grammar checking and academic style.

Conflict of interest There are no relevant financial or non-financial competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Micali, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., & Sutton, C. (2021). Program synthesis with large language models. arXiv preprint arXiv:2108.07732 <https://doi.org/10.48550/arXiv.2108.07732>
- Akyash M, Kamali HM (2025) SimEval: Investigating the similarity obstacle in LLM-based hardware code generation. Proceedings of the Asia and South Pacific Design Automation Conference, ASP-DAC, 1002–1007. <https://doi.org/10.1145/3658617.3697624>
- Bai X, Huang SB, Wei C, Wang R, Tong Y, Li L, Wu Y, Zhao C (2025) Collaboration between intelligent agents and large language models: A novel approach for enhancing code generation capability. Expert Syst Appl 269:126357. <https://doi.org/10.1016/j.eswa.2024.126357>
- Black GS, Rimal BP, Vaidyan VM (2025) Balancing security and correctness in code generation: An empirical study on commercial large language models. IEEE Trans Emerg Top Comput Intell 9(1):419–430. <https://doi.org/10.1109/TETCI.2024.3446695>
- Bose DB (2025) From prompts to properties: Rethinking LLM code generation with property-based testing. Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering, 1660–1665. <https://doi.org/10.1145/3696630.3728702>
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., & Zaremba, W. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 <https://doi.org/10.48550/arXiv.2107.03374>
- Christakis N, Drikakis D (2025) Evaluating large language models in code generation: INFINITE methodology for defining the inference index. Appl Sci 15(7):3784. <https://doi.org/10.3390/ap15073784>
- Djordjevic V, Dubonjic L, Morato MM, Prsic D, Stojanovic V (2022) Sensor fault estimation for hydraulic servo actuator based on sliding mode observer. Math Modelling Control 2(1):34–43. <https://doi.org/10.3934/mmc.2022005>
- Dora S, Lunkad D, Aslam N, Venkatesan S, Shukla SK (2025) The hidden risks of LLM-generated web application code: A security-centric evaluation of code generation capabilities in large language models. In *Information Systems Security, Lecture Notes in Computer Science* (pp. 27–37). Springer. https://doi.org/10.1007/978-3-032-13714-2_3
- Du M, Luu AT, Ji B, Wu X, Huang D, Qing Y, Zhuo TY, Liu Q, Ng SK (2025) CodeArena: A collective evaluation platform for LLM code generation. Proceedings of the Annual Meeting of the Association for Computational Linguistics, 3, 502–512. <https://doi.org/10.18653/v1/2025.acl-demo.48>
- Fakhoury S, Naik A, Sakkas G, Chakraborty S, Lahiri SK (2024) LLM-based test-driven interactive code generation: User study and empirical evaluation. IEEE Trans Software Eng 50(9):2254–2268. <https://doi.org/10.1109/TSE.2024.3428972>
- Han H, Kim J, Yoo J, Lee Y, Hwang S (2024) ARCHCODE: Incorporating software requirements in code generation with large language models. Proc Annual Meeting Association Comput Linguistics 1:13520–13552. <https://doi.org/10.18653/v1/2024.acl-long.730>
- He X, Li D, Wen H, Zhu Y, Liu C, Yan M, Zhang H (2025) CoSEFA: An LLM-based programming assistant for secure code generation via supervised co-decoding. Proc ACM SIGSOFT Symp Found Softw Eng 1198:1202. <https://doi.org/10.1145/3696630.3728609>
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, J., Song, D., & Steinhardt, J. (2021). Measuring coding challenge competence with APPS. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks <https://doi.org/10.48550/arXiv.2105.09938>
- Hou S, Zhao A, Liang J, Shen Z, Wu H (2025) Geo-FuB: A method for constructing an operator-function knowledge base for geospatial code generation with large language models. Knowl Based Syst 319:113624. <https://doi.org/10.1016/j.knsys.2025.113624>
- Iqbal I, Walayat K, Kakar MU, Ma J (2022) Automated identification of human gastrointestinal tract abnormalities based on deep convolutional neural network with endoscopic images. Intell Syst Appl 16:200149. <https://doi.org/10.1016/j.iswa.2022.200149>
- Iqbal I, Ullah I, Peng T, Wang W, Ma N (2025) An end-to-end deep convolutional neural network-based data-driven fusion framework for identification of human induced pluripotent stem cell-derived endothelial cells in photomicrographs. Eng Appl Artif Intell 139:109573. <https://doi.org/10.1016/j.engappai.2024.109573>
- Ji D, Ren S, Zheng R, Yi R, Zhao H, Yang L (2013) A tracking control method of ASV following AUV. In *2013 OCEANS - San Diego* (pp. 1–6). IEEE
- Jiang X, Dong Y, Wang L, Fang Z, Shang Q, Li G, Jin Z, Jiao W (2024) Self-planning code generation with large language models. ACM Trans Softw Eng Methodol 33(7):182. <https://doi.org/10.1145/3672456>
- Jiang X, Dong Y, Tao Y, Liu H, Jin Z, Li G (2025) ROCODE: Integrating backtracking mechanism and program analysis in

- large language models for code generation. *Proc - Int Conf Softw Eng* 334–346. <https://doi.org/10.1109/ICSE55347.2025.00133>
21. Lei R, Wang H, Dong J, Wang H, Liu S, Laili Y, Zhang L (2025) MetaIndux-PLC: A control logic-guided LLM for PLC code generation in industrial control systems. *Appl Soft Comput* 184:113673. <https://doi.org/10.1016/j.asoc.2025.113673>
 22. Li Y, Lyu C (2025) Symmetry-aware code generation: Distilling pseudocode reasoning for lightweight deployment of large language models. *Symmetry* 17(8):1325. <https://doi.org/10.3390/sym17081325>
 23. Li Y, Huang S, Wei C, Wang R, Wang X, Tong Y, Li L, Wu Y, Zhao C (2024) Chain-of-thought in neural code generation: From and for lightweight language models. *IEEE Trans Software Eng* 50(9):2437–2457. <https://doi.org/10.1109/TSE.2024.3440503>
 24. Li J, Tao C, Li G, Jin Z, Zhang H, Fang Z, Liu F (2025) Large language model-aware in-context learning for code generation. *ACM Trans Softw Eng Methodol* 34(7):190. <https://doi.org/10.1145/3715908>
 25. Lin F, Kim D, Chen T (2025) SOEN-101: Code generation by emulating software process models using large language model agents. *Proceedings - International Conference on Software Engineering*, 1527–1539. <https://doi.org/10.1109/ICSE55347.2025.00140>
 26. Newcomb L, Newcomb A, Ochoa O (2025) Preconditions and postconditions as design constraints for LLM code generation. *IEEE Access* 13:184259–184274. <https://doi.org/10.1109/ACCESS.2025.3625819>
 27. Nyamsuren E (2025) Evaluating quantized large language models for code generation on low-resource language benchmarks. *J Comput Lang* 84:101351. <https://doi.org/10.1016/j.cola.2025.101351>
 28. Pendyala VS, Thakur NB (2025) Performance and interpretability analysis of code generation large language models. *Neurocomputing* 656:131461. <https://doi.org/10.1016/j.neucom.2025.131461>
 29. Pinckney N, Batten CF, Liu M, Ren H, Khailany BK (2025) Revisiting VerilogEval: A year of improvements in large-language models for hardware code generation. *ACM Trans Des Autom Electron Syst* 30(6):ART91. <https://doi.org/10.1145/3718088>
 30. Qu Y, Huang S, Li Y, Bai T, Chen X, Wang X, Li L, Yao Y (2025) BadCodePrompt: Backdoor attacks against prompt engineering of large language models for code generation. *Automated Softw Eng* 32(1):17. <https://doi.org/10.1007/s10515-024-00485-2>
 31. Riddell M, Ni A, Cohan A (2024) Quantifying contamination in evaluating code generation capabilities of language models. *Proc Annual Meeting Association Comput Linguistics* 1:14116–14137. <https://doi.org/10.18653/v1/2024.acl-long.761>
 32. Soliman AS, Shaheen SI, Hadhoud MM (2024) Leveraging pre-trained language models for code generation. *Complex Intell Syst* 10(3):3955–3980. <https://doi.org/10.1007/s40747-024-01373-8>
 33. Song X, Peng Z, Song S, Stojanovic V (2024) Interval observer design for unobservable switched nonlinear partial differential equation systems and its application. *Int J Robust Nonlinear Control* 34(16):10990–11009. <https://doi.org/10.1002/rnc.7553>
 34. Tao X, Lang N, Li H, Xu D (2022) Path planning in uncertain environment with moving obstacles using warm start cross entropy. *IEEE/ASME Trans Mechatron* 27(2):800–810. <https://doi.org/10.1109/TMECH.2021.3071723>
 35. Thakur S, Ahmad B, Pearce HA, Tan B, Dolan-Gavitt B, Karri R, Garg SJ (2024) VeriGen: A large language model for Verilog code generation. *ACM Trans Des Autom Electron Syst* 29(3):46. <https://doi.org/10.1145/3643681>
 36. Tian Z, Chen J, Zhang X, Dai H (2025) Fixing large language models' specification misunderstanding for better code generation. *Proc - Int Conf Softw Eng* 1514–1526. <https://doi.org/10.1109/ICSE55347.2025.00108>
 37. Wang Z, Zhou Z, Song D, Huang Y, Chen S, Ma L, Zhang T (2025) Towards understanding the characteristics of code generation errors made by large language models. *Proc - Int Conf Softw Eng* 2587–2599. <https://doi.org/10.1109/ICSE55347.2025.00180>
 38. Weyssow M, Zhou X, Kim K, Lo D, Sahraoui HA (2025) Exploring parameter-efficient fine-tuning techniques for code generation with large language models. *ACM Trans Softw Eng Methodol* 34(7):204. <https://doi.org/10.1145/3714461>
 39. Wu H, Shen Z, Hou S, Jiao H, Liu Z, Xie L, Liang J, Qing Y, Zhang X, Gui Z, Guan X (2025) AutoGEEval++: A multi-level and multi-geospatial-modality automated evaluation framework for large language models in geospatial code generation on Google Earth Engine. *Big Earth Data*. <https://doi.org/10.1080/20964471.2025.2581425>
 40. Wu H, Jiao H, Hou S, Liang J, Shen Z, Zhao A, Qing Y, Jin F, Guan X, Gui Z (2025a) GeoColab: An LLM-based multi-agent collaborative framework for geospatial code generation. *Int J Digit Earth* 18(2):2569405. <https://doi.org/10.1080/17538947.2025.2569405>
 41. Yang C, Yu L, Su H, Zhou X (2025) APICoder: A multi-role large language model framework for API service call code generation. *Proc IEEE Int Conf Web Serv ICWS*:849–851. <https://doi.org/10.1109/ICWS67624.2025.00109>
 42. Zan D, Chen B, Gong Y, Cao J, Zhang F, Wu B, Guan B, Yin Y, Wang Y (2025) Private-library-oriented code generation with large language models. *Knowl Based Syst* 326:113934. <https://doi.org/10.1016/j.knosys.2025.113934>
 43. Zhang X, Zhou Y, Yang G, Gall HC, Chen T (2025) Anchor attention, small cache: Code generation with large language models. *IEEE Trans Software Eng* 51(6):1866–1881. <https://doi.org/10.1109/TSE.2025.3570680>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.